

Polimorfismo

Clase 6

Características y conceptos de POO

Estos conceptos facilitan el mantenimiento y la reutilización del código.

- Herencia
- Acoplamiento
- Encapsulamiento
- Cohesión
- Polimorfismo

Herencia

La herencia es un mecanismo para **reutilizar código**. Permite a los desarrolladores crear nuevas **clases que se derivan** de clases existentes. Las nuevas clases, conocidas como clases hijas o subclases, **heredan los atributos y métodos** de la clase padre, lo que puede ahorrar tiempo y reducir la redundancia en el código.

Encapsulamiento

El encapsulamiento es un principio de la POO que consiste en **ocultar los detalles internos** de un objeto y proporcionar una interfaz controlada para interactuar con él. Esto se logra mediante la creación de **atributos privados** y el acceso a ellos a través de métodos públicos (**getters y setters**). El encapsulamiento **proporciona control** sobre el acceso a los datos y **protege la integridad** de un objeto, lo que facilita el mantenimiento y la evolución del código.

Acoplamiento

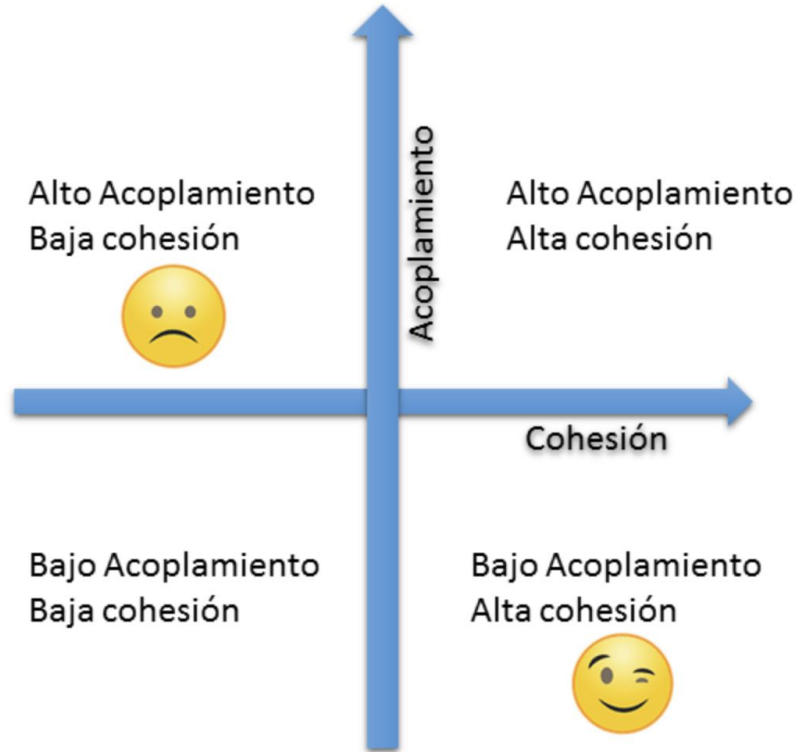
El acoplamiento se refiere al grado de dependencia entre los módulos o componentes de un sistema de software. Un **acoplamiento bajo es deseable**, ya que indica que los componentes **son independientes** y pueden cambiar sin afectar en gran medida a otros componentes. Un **acoplamiento alto**, por otro lado, indica una **fuerte dependencia** entre los componentes, lo que hace que el sistema sea más rígido y difícil de mantener. La POO busca reducir el acoplamiento entre clases, lo que **favorece la modularidad y la flexibilidad del código**.

Cohesión

La cohesión se refiere a la medida en que los miembros (métodos y atributos) de una clase están relacionados y trabajan juntos para **cumplir una función específica**.

La alta cohesión es un objetivo deseable en la POO, ya que conduce a **clases más específicas y reutilizables**.

Acoplamiento vs Cohesión



Polimorfismo

El polimorfismo es un concepto que permite que los objetos de **diferentes clases** respondan de manera única a las **mismas llamadas a métodos**. Esto significa que objetos de distintas clases pueden implementar un método de manera específica para su tipo, lo que permite una flexibilidad y adaptabilidad en el diseño de software. El polimorfismo se logra a través de la herencia y la implementación de **métodos con el mismo nombre en diferentes clases**, pero con **comportamientos específicos** para cada una.

Ejemplo de polimorfismo

```
class Animal:  
    def hacer_sonido(self):  
        pass
```

```
class Perro(Animal):  
    def hacer_sonido(self):  
        return "Guau"
```

```
class Gato(Animal):  
    def hacer_sonido(self):  
        return "Miau"
```

```
class Pajaro(Animal):  
    def hacer_sonido(self):  
        return "Cantar"
```

```
# Uso de polimorfismo para hacer sonidos  
animales = [Perro(), Gato(), Pajaro()]
```

```
for animal in animales:  
    print(f"Sonido: {animal.hacer_sonido()}")
```

Polimorfismo (ejemplo)

```
class Vehiculo:
    def calcular_costo_seguro(self):
        pass

class Auto(Vehiculo):
    def calcular_costo_seguro(self):
        # Cálculo específico para coche
        return 1000

class Moto(Vehiculo):
    def calcular_costo_seguro(self):
        # Cálculo específico para moto
        return 800

class Bicicleta(Vehiculo):
    def calcular_costo_seguro(self):
        # Cálculo específico para bicicleta
        return 200
```

Uso del polimorfismo

```
vehiculos = [Auto(), Moto(), Bicicleta()]
```

Si no implementé el metodo calcular_costo
for vehiculo in vehiculos:

```
    if isinstance(vehiculo, Auto):
        print(1000)
    elif isinstance(vehiculo, Moto):
        print(800)
    else:
        print(200)
```

Polimorfismo

```
for vehiculo in vehiculos:
    print(vehiculo.calcular_costo_seguro())
```

Recordar

Revisar los métodos **isinstance(obj, clase)** y **issubclass(clase1, clase2)**.

Se puede sugerir el tipo del parámetro que se espera recibir desde una función.
Pero esto no es obligatorio, ni provoca alguna excepción si no se cumple.

Ej:

```
def funcion(listado: list, nombre: str, edad: int) -> None:
```

```
    pass
```

Ejercicio

Crea una jerarquía de clases para representar figuras geométricas como círculos (radio), rectángulos (base, altura) y triángulos (base, altura). Cada clase debe tener un método para calcular su área. Luego, utiliza el polimorfismo para calcular y mostrar el área de varias figuras geométricas sin conocer su tipo específico.

Ejercicio anterior

Se cuenta con una clase CuentaBancaria de la cual se deben crear 2 clases derivadas. Una para las cuentas de débito y otra para las de crédito. Ambas deben ser creadas con un saldo de 0 y 10000 pesos respectivamente. Se deben implementar los métodos de retirar y depositar montos solo si la contraseña es correcta y si cuenta con el saldo suficiente.

Por cada retiro las cuentas de débito tienen un recargo de 10 pesos. Las cuentas de crédito tienen un límite, solo se puede retirar como máximo 1000 pesos por cada retiro.

Continuación

Del ejercicio anterior crear una clase que represente a un banco. El banco cuenta con nombre, dirección, teléfono, dinero para mantenimiento y una lista de cuentas bancarias.

Se desean implementar las siguientes funcionalidades:

- Anualmente se deberá descontar un recargo de 100 pesos por mantenimientos a todas las cuentas por igual.
- Al mes se quiere aplicar una bonificación mensual a todas las cuentas de un banco. A las de débito se les aumenta el 0.5% de su saldo actual. En cambio a las de crédito se les deberá aumentar el saldo en 1% si no tienen mora, en caso contrario se deberá informar el nombre del titular con mora.
- Anualmente se quiere aumentar o disminuir el tope de crédito en 10000 pesos si es que la cuenta de crédito posee o no mora respectivamente en la actualidad.
- Implementar una función que imprima los nombres y saldos actuales de todas las cuentas del banco.
- Dar de alta una cuenta bancaria. Y eliminar una cuenta dado un dni y contraseña.